SERVER SCRIPT

DELIVERY NOTE BARCODE API

Script Type: API

Reference Document Type: Delivery Note

Event Frequency: All

DocType Event: Before Insert

Script

```
"""
Delivery Note Barcode API - Server Script
Script Type: API
Reference Document Type: (Leave blank for API type)

This provides API endpoints for barcode scanning functionality
"""

import frappe
from frappe import _
import json


@frappe.whitelist()
def process_barcode_scan(barcode, delivery_note=None):
    """
    Process barcode scan and return complete item and batch details

    Args:
        barcode: Scanned barcode value
        delivery_note: Delivery Note name (optional)

    Returns:
        dict: Item details including batch info and pricing
    """
    if not barcode:
        frappe.throw(_("Barcode is required"))

    result = {
        'success': False,
        'item_code': None,
        'batch_no': None,
        'item_details': {},
        'batch_details': {},
        'message': ''
    }
```

```
# First, try to find batch directly
batch = frappe.db.get_value(
    'Batch',
    barcode,
    ['name', 'item', 'batch_qty', 'expiry_date', 'manufacturing_date'],
    as_dict=1
)

if batch:
    result['success'] = True
    result['batch_no'] = batch.name
    result['item_code'] = batch.item
    result['batch_details'] = {
        'batch_no': batch.name,
        'batch_qty': batch.batch_qty,
        'expiry_date': batch.expiry_date,
        'manufacturing_date': batch.manufacturing_date
    }
    result['message'] = f"Batch {batch.name} found"

    # Get item details
    item_details = get_item_details_for_delivery(
        batch.item,
        batch.name,
        delivery_note
    )
    result['item_details'] = item_details

    return result

# If not found as batch, try as item barcode
item_barcode = frappe.db.get_value(
    'Item Barcode',
    {'barcode': barcode},
    ['parent', 'barcode'],
    as_dict=1
)

if item_barcode:
    result['success'] = True
    result['item_code'] = item_barcode.parent
    result['message'] = f"Item {item_barcode.parent} found by barcode"

    # Check if item has batch
    has_batch = frappe.db.get_value('Item', item_barcode.parent, 'has_batch_no')

    if has_batch:
```

```
    if has_batch:
        # Try to find a batch for this item with the barcode
        batch = frappe.db.get_value(
            'Batch',
            {'item': item_barcode.parent, 'name': barcode},
            ['name', 'batch_qty', 'expiry_date', 'manufacturing_date'],
            as_dict=1
        )

        if batch:
            result['batch_no'] = batch.name
            result['batch_details'] = {
                'batch_no': batch.name,
                'batch_qty': batch.batch_qty,
                'expiry_date': batch.expiry_date,
                'manufacturing_date': batch.manufacturing_date
            }

    # Get item details
    item_details = get_item_details_for_delivery(
        item_barcode.parent,
        result.get('batch_no'),
        delivery_note
    )
    result['item_details'] = item_details

    return result

# Try serial number
serial = frappe.db.get_value(
    'Serial No',
    barcode,
    ['name', 'item_code', 'batch_no'],
    as_dict=1
)

if serial:
    result['success'] = True
    result['item_code'] = serial.item_code
    result['batch_no'] = serial.batch_no
    result['message'] = f"Serial No {serial.name} found"

    if serial.batch_no:
        batch_details = frappe.db.get_value(
            'Batch',
            serial.batch_no,
            ['batch_qty', 'expiry_date', 'manufacturing_date'],
```

```
            as_dict=1
        )
        if batch_details:
            result['batch_details'] = {
                'batch_no': serial.batch_no,
                'batch_qty': batch_details.batch_qty,
                'expiry_date': batch_details.expiry_date,
                'manufacturing_date': batch_details.manufacturing_date
            }

    # Get item details
    item_details = get_item_details_for_delivery(
        serial.item_code,
        serial.batch_no,
        delivery_note
    )
    result['item_details'] = item_details

    return result

# Nothing found
result['message'] = f"No item, batch, or serial found for barcode: {barcode}"
return result


def get_item_details_for_delivery(item_code, batch_no=None, delivery_note=None):
    """
    Get complete item details including pricing
    """
    from erpnext.stock.get_item_details import get_item_details

    args = {
        'item_code': item_code,
        'doctype': 'Delivery Note',
        'qty': 1,
        'stock_qty': 1,
    }

    if delivery_note:
        dn = frappe.get_doc('Delivery Note', delivery_note)
        args.update({
            'company': dn.company,
            'customer': dn.customer,
            'price_list': dn.selling_price_list,
            'currency': dn.currency,
            'conversion_factor': 1,
```

```
            'warehouse': dn.set_warehouse,
            'transaction_date': dn.posting_date,
            'ignore_pricing_rule': dn.ignore_pricing_rule,
            'name': dn.name,
        })
    else:
        default_company = frappe.defaults.get_user_default("Company")
        args.update({
            'company': default_company,
            'transaction_date': frappe.utils.today(),
        })

    if batch_no:
        args['batch_no'] = batch_no

    item_details = get_item_details(args)

    if batch_no and delivery_note:
        try:
            batch_price = get_batch_price_safe(item_code, batch_no, delivery_note)
            if batch_price:
                item_details['price_list_rate'] = batch_price
                item_details['rate'] = batch_price
        except Exception as e:
            frappe.log_error(f"Error getting batch price: {str(e)}", "Batch Price Error")

    return item_details


def get_batch_price_safe(item_code, batch_no, delivery_note):
    """
    Safely get batch-based price with proper error handling
    """
    try:
        dn = frappe.get_doc('Delivery Note', delivery_note)

        pctx = {
            'items': dn.items,
            'currency': dn.currency,
            'conversion_rate': dn.conversion_rate or 1,
            'price_list': dn.selling_price_list,
            'price_list_currency': dn.price_list_currency,
            'plc_conversion_rate': dn.plc_conversion_rate or 1,
            'company': dn.company,
            'transaction_date': dn.posting_date,
            'ignore_pricing_rule': dn.ignore_pricing_rule or 0,
            'doctype': 'Delivery Note'
```

```
            'doctype': 'Delivery Note',
            'name': dn.name,
            'is_return': dn.is_return or 0,
            'update_stock': dn.update_stock or 0,
            'pos_profile': '',
            'is_internal_customer': dn.is_internal_customer or 0,
            'batch_no': batch_no
        }

        from erpnext.stock.get_item_details import get_batch_based_item_price
        result = get_batch_based_item_price(item_code, pctx)

        if result and isinstance(result, dict) and result.get('price_list_rate'):
            return result.get('price_list_rate')

    except Exception as e:
        frappe.log_error(f"Batch price error: {str(e)}", "Get Batch Price")

    return None


@frappe.whitelist()
def get_batch_details(batch_no, item_code=None):
    """
    Get detailed batch information
    """
    filters = {'name': batch_no}
    if item_code:
        filters['item'] = item_code

    batch = frappe.db.get_value(
        'Batch',
        filters,
        ['name', 'item', 'batch_qty', 'expiry_date', 'manufacturing_date', 'description'],
        as_dict=1
    )

    if not batch:
        if item_code:
            frappe.throw(_("Batch {0} does not exist for item {1}").format(batch_no, item_code))
        else:
            frappe.throw(_("Batch {0} does not exist").format(batch_no))

    from erpnext.stock.doctype.batch.batch import get_batch_qty
    batch['available_qty'] = get_batch_qty(batch_no)

    return batch
```

```
@frappe.whitelist()
def validate_batch_item(batch_no, item_code):
    """
    Validate if batch belongs to item
    """
    batch_item = frappe.db.get_value('Batch', batch_no, 'item')

    if not batch_item:
        return {
            'valid': False,
            'message': f"Batch {batch_no} does not exist"
        }

    if batch_item != item_code:
        return {
            'valid': False,
            'message': f"Batch {batch_no} belongs to {batch_item}, not {item_code}"
        }

    return {
        'valid': True,
        'message': 'Valid'
    }
```

Request Limit:	5
Time Window (Seconds):	86400